AWS VPC PEERING PROJECT

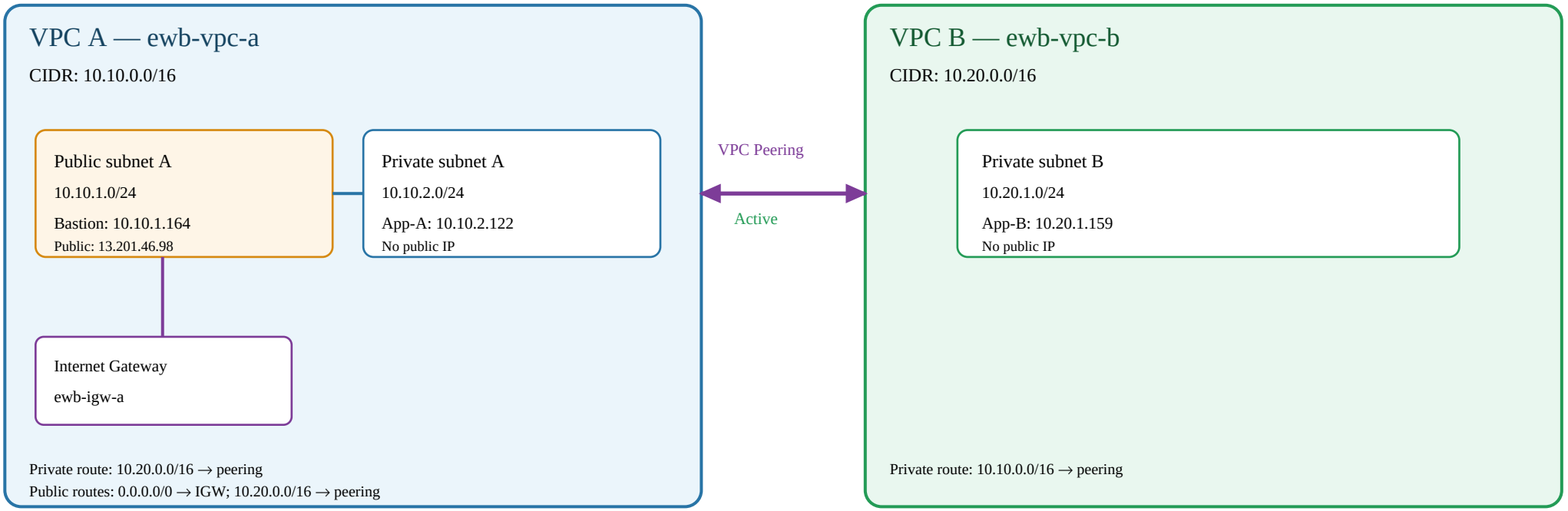
EWB Cloud Labs — Student Project Report

Student Name	Sriram Sandhya
Batch	AWS Internship Batch-1
Project Title	AWS VPC Peering
AWS Region	Asia Pacific (Mumbai) — ap-south-1
Completion Date	21 October 2026

Purpose: connect two private IPv4 VPC networks using VPC peering, keep application instances private, and use a bastion host for administration.

AWS VPC Peering Project

Architecture Diagram & Resource Inventory | Region: Asia Pacific (Mumbai), ap-south-1



Resource Inventory (known actual values)

Resource	Name	CIDR / Private IP	Public IP / Resource ID	Notes
VPC A	ewb-vpc-a	10.10.0.0/16	vpc-0cde555b96d6aa54b	Mumbai region
VPC B	ewb-vpc-b	10.20.0.0/16	ADD actual VPC ID	Get from VPC console
Public subnet A	ewb-a-public	10.10.1.0/24	ADD actual subnet ID	VPC A
Private subnet A	ewb-a-private	10.10.2.0/24	ADD actual subnet ID	VPC A
Private subnet B	ewb-b-private	10.20.1.0/24	ADD actual subnet ID	VPC B
Bastion EC2	ewb-bastion	10.10.1.164	13.201.46.98	Public subnet; key pair ewb-lab
App-A EC2	ewb-app-a	10.10.2.122	None	Private instance
App-B EC2	ewb-app-b	10.20.1.159	None	Private instance
Internet Gateway	ewb-igw-a	—	ADD actual IGW ID	Attached to VPC A
VPC Peering	ewb-a-b-peer	10.10.0.0/16 ↔ 10.20.0.0/16	ADD actual pcx ID	Reported Active
Route table	ewb-rt-a-public	0.0.0.0/0 → IGW; 10.20.0.0/16 → pcx	ADD actual route table ID	Public subnet A association

Resource	Name	CIDR / Private IP	Public IP / Resource ID	Notes
Route table	ewb-rt-a-private	10.20.0.0/16 → pcx	ADD actual route table ID	Private subnet A association
Route table	ewb-rt-b-private	10.10.0.0/16 → pcx	ADD actual route table ID	Private subnet B association

Before submitting: Replace each “ADD actual … ID” placeholder using the AWS Console. Confirm peering status and subnet associations in your screenshots. Do not include private keys, passwords, or access tokens.

1. Overview and Architecture

Two VPCs are configured in the Mumbai region. VPC A contains a public subnet for the bastion host and a private subnet for App-A. VPC B contains a private subnet for App-B. VPC peering provides private routing between their CIDR ranges.

VPC A — 10.10.0.0/16	VPC Peering	VPC B — 10.20.0.0/16
Public subnet 10.10.1.0/24	■	Private subnet 10.20.1.0/24
Bastion 10.10.1.164; public 13.201.46.98	■	App-B 10.20.1.159; private only
Private subnet 10.10.2.0/24		App-A 10.10.2.122; private only

Actual resource inventory

Resource	Name / ID / CIDR
VPC A	ewb-vpc-a — vpc-0cde555b96d6aa54b — 10.10.0.0/16
VPC B	ewb-vpc-b — vpc-0039bf52224874dec — 10.20.0.0/16
Subnet A public	ewb-a-public — 10.10.1.0/24
Subnet A private	ewb-a-private — 10.10.2.0/24
Subnet B private	ewb-b-private — 10.20.1.0/24
Internet Gateway	ewb-igw-a — igw-074da52de572e662d; detached during cleanup
VPC Peering	ewb-a-b-peer — pcx-060872035e3f181e0; deleted during cleanup
Route tables	ewb-rt-a-public; ewb-rt-a-private; ewb-rt-b-private
Bastion EC2	ewb-bastion — i-04fbc9405b2215885 — 10.10.1.164 — public 13.201.46.98
App-A EC2	ewb-app-a — i-0069c9d7869ac1b58 — 10.10.2.122 — no public IPv4 shown
App-B EC2	ewb-app-b — i-0d1449caaf9fa7f6d — 10.20.1.159 — no public IPv4 shown

2. Network Configuration

Subnet CIDRs: ewb-a-public 10.10.1.0/24; ewb-a-private 10.10.2.0/24; ewb-b-private 10.20.1.0/24.

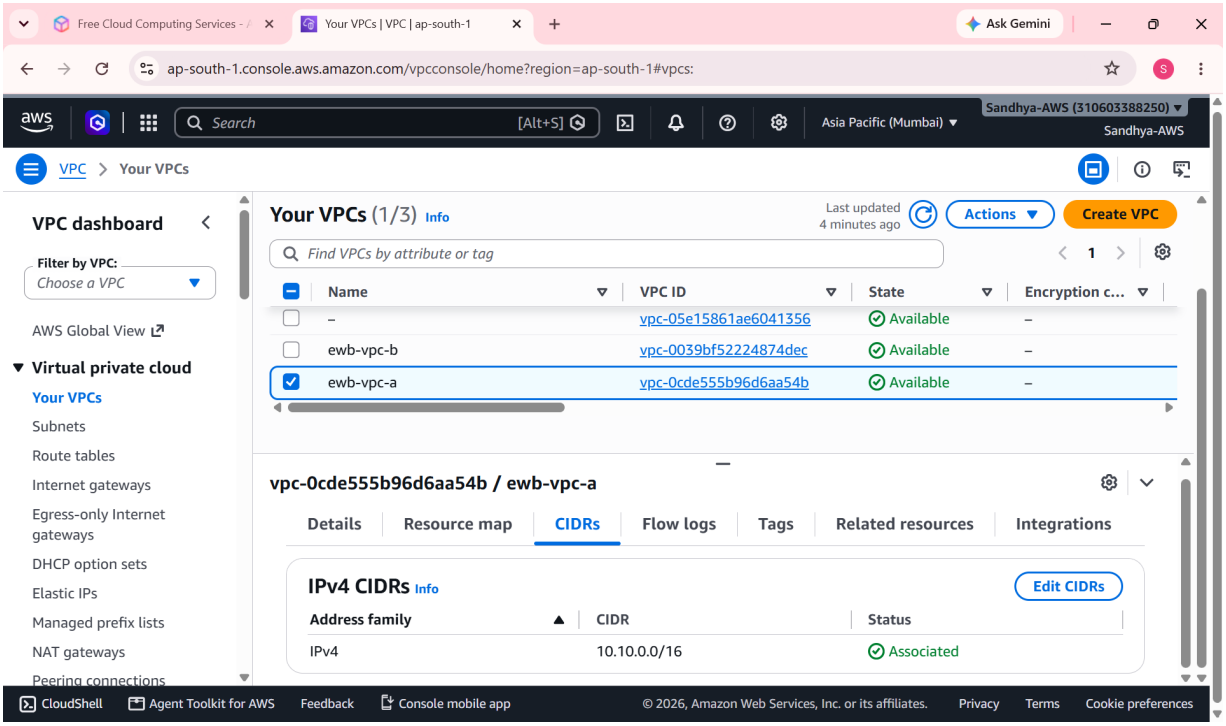
Route table	Subnet association	Routes
ewb-rt-a-public	ewb-a-public	10.10.0.0/16 → local; 0.0.0.0/0 → IGW; 10.20.0.0/16 → peering
ewb-rt-a-private	ewb-a-private	10.10.0.0/16 → local; 10.20.0.0/16 → peering
ewb-rt-b-private	ewb-b-private	10.20.0.0/16 → local; 10.10.0.0/16 → peering

The peering connection was Active during testing. Cleanup evidence later shows it deleted. The Internet Gateway was detached, and a deletion confirmation screen was captured.

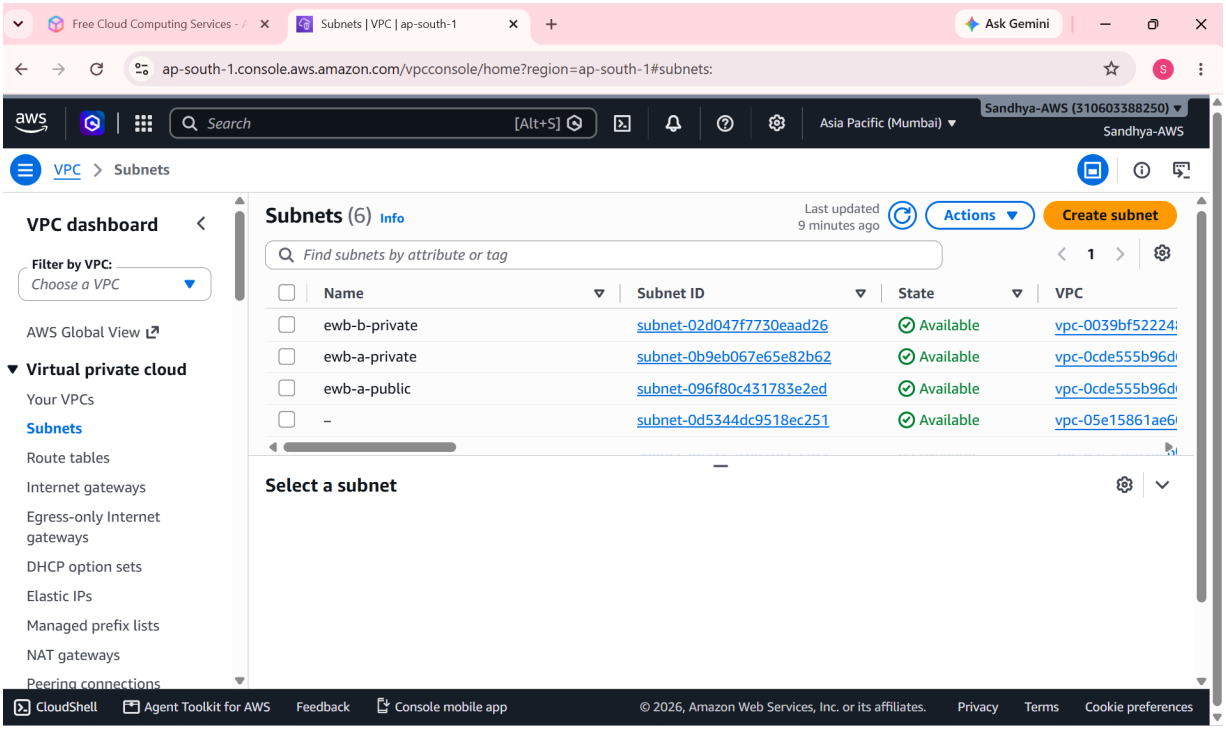
Security group inbound rules

Bastion: SSH TCP/22 from 106.200.30.98/32. App-A: SSH from 10.10.1.164/32 and ICMP rules shown from 10.20.1.159/32 and 10.10.1.164/32. App-B: SSH from 10.10.1.164/32 and ICMP rules shown from 10.10.2.122/32 and 10.10.1.164/32. Rules are summarized as visible in the uploaded screenshots.

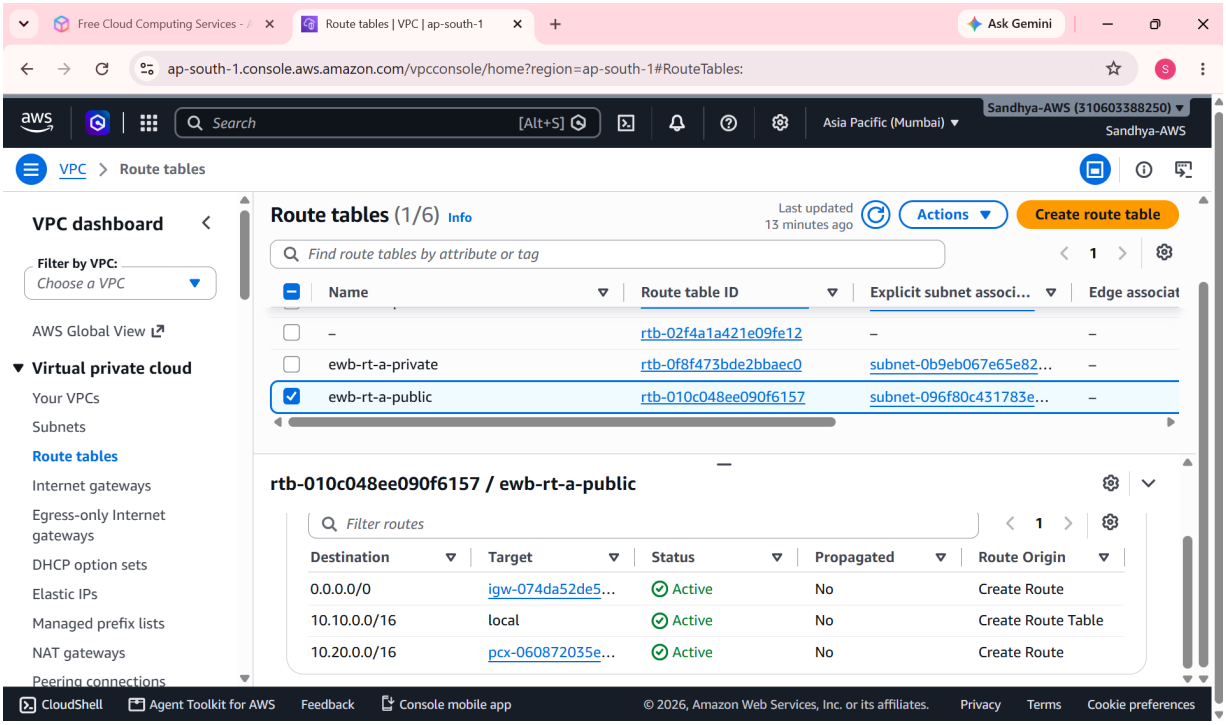
Evidence 1 — VPC A CIDR and resource ID.



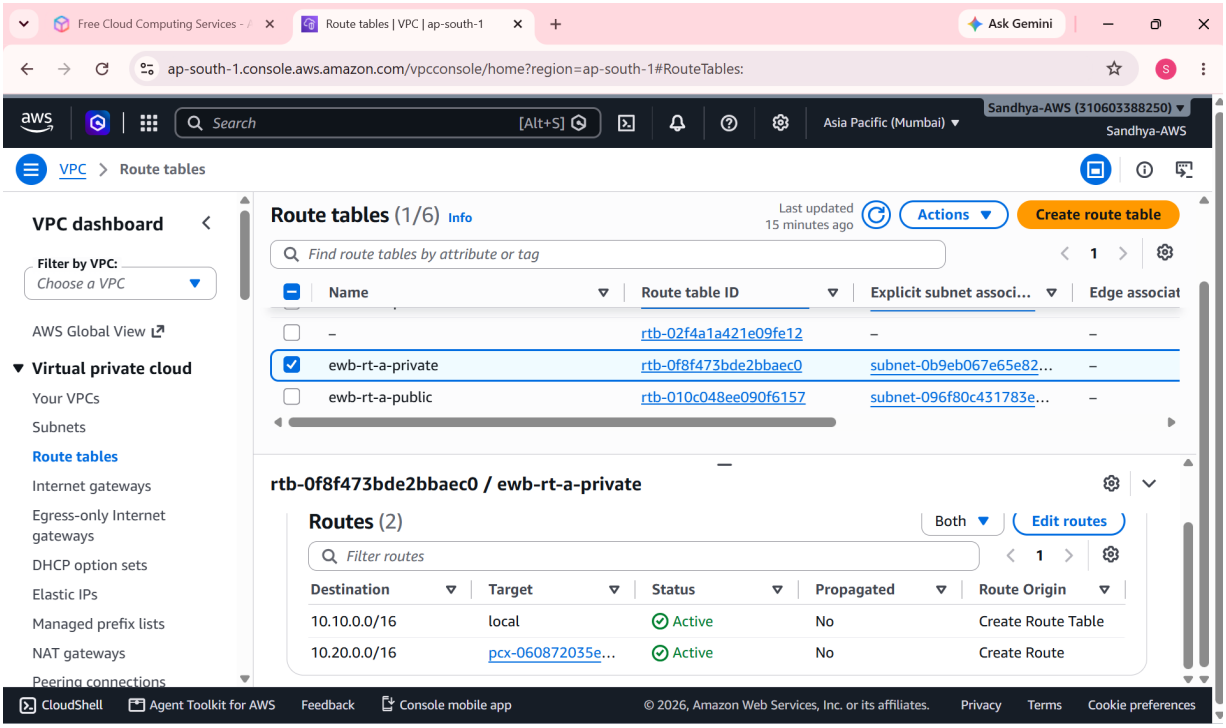
Evidence 3 — Subnet inventory.



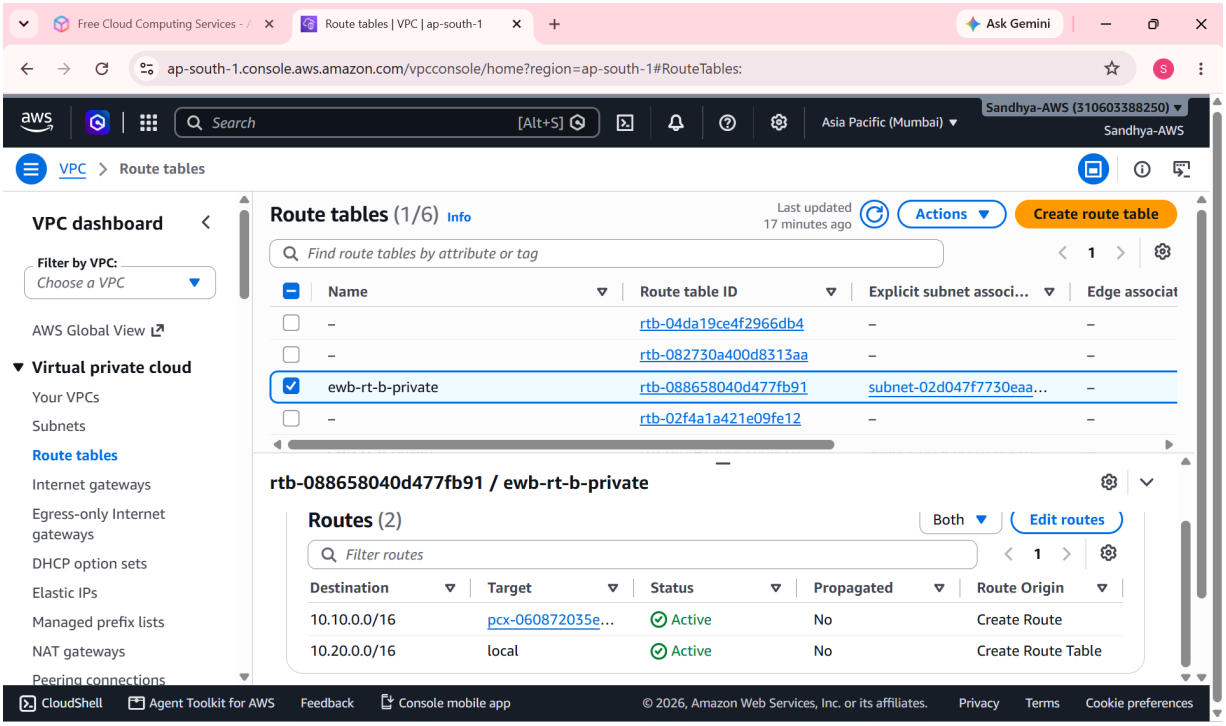
Evidence 4 — Public route table.



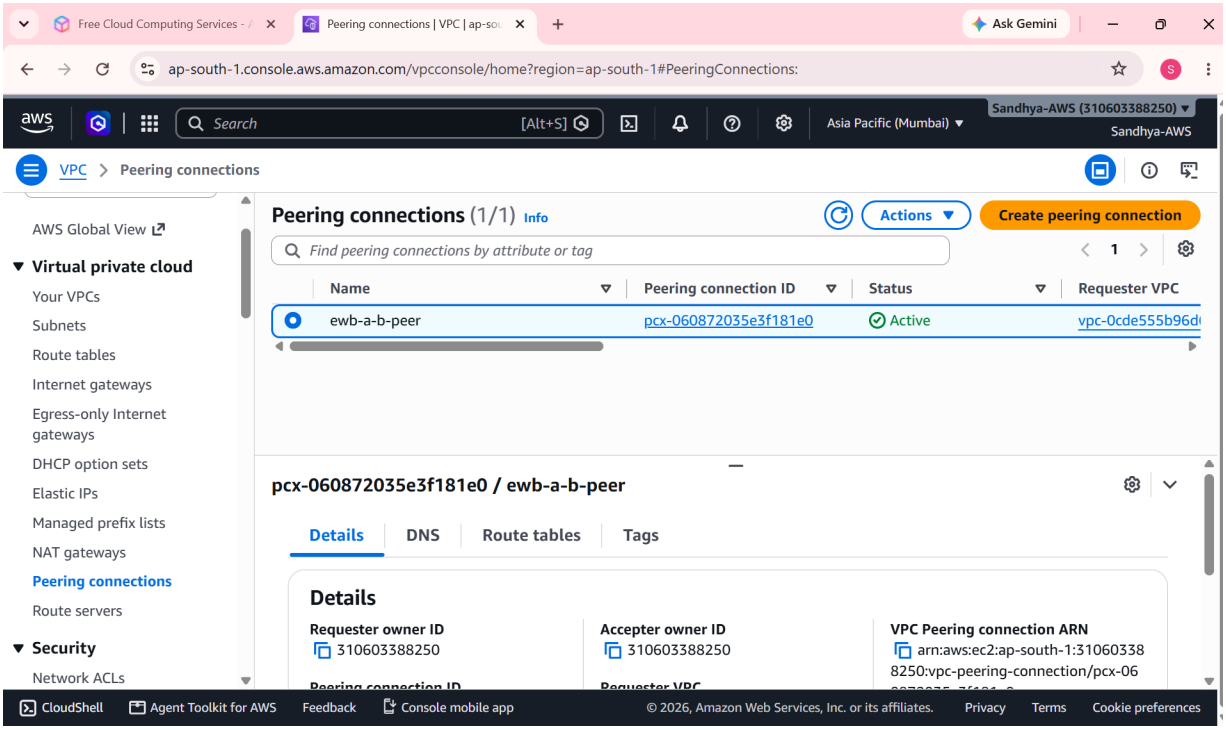
Evidence 5 — Private route table A.



Evidence 6 — Private route table B.



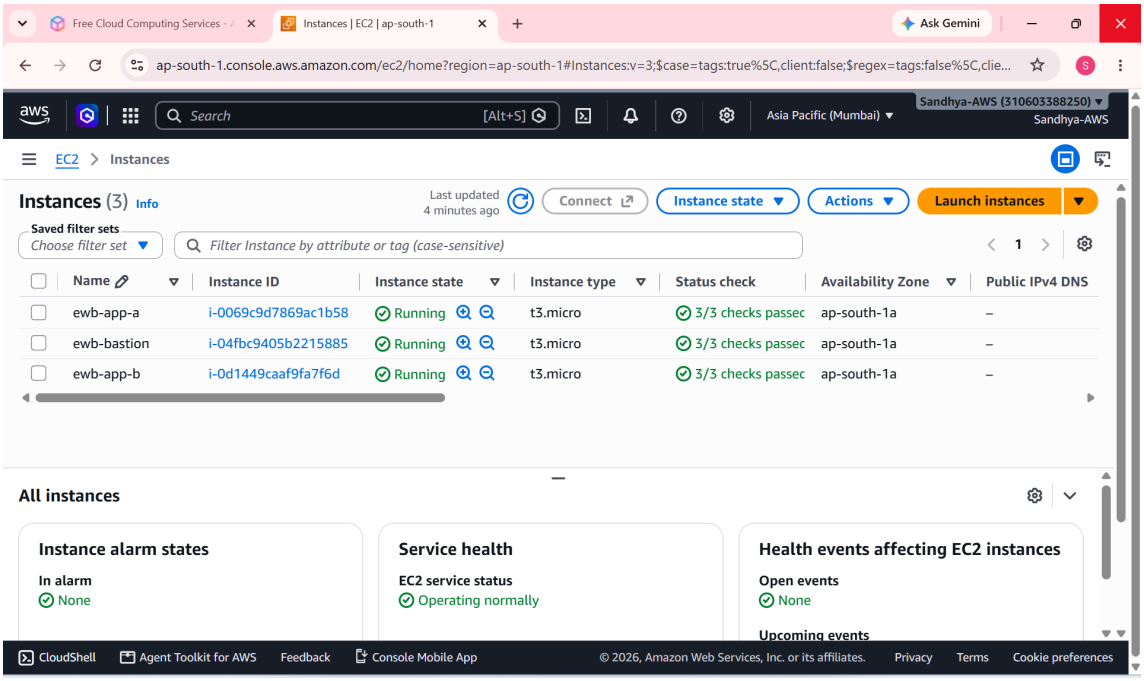
Evidence 7 — Peering Active during testing.



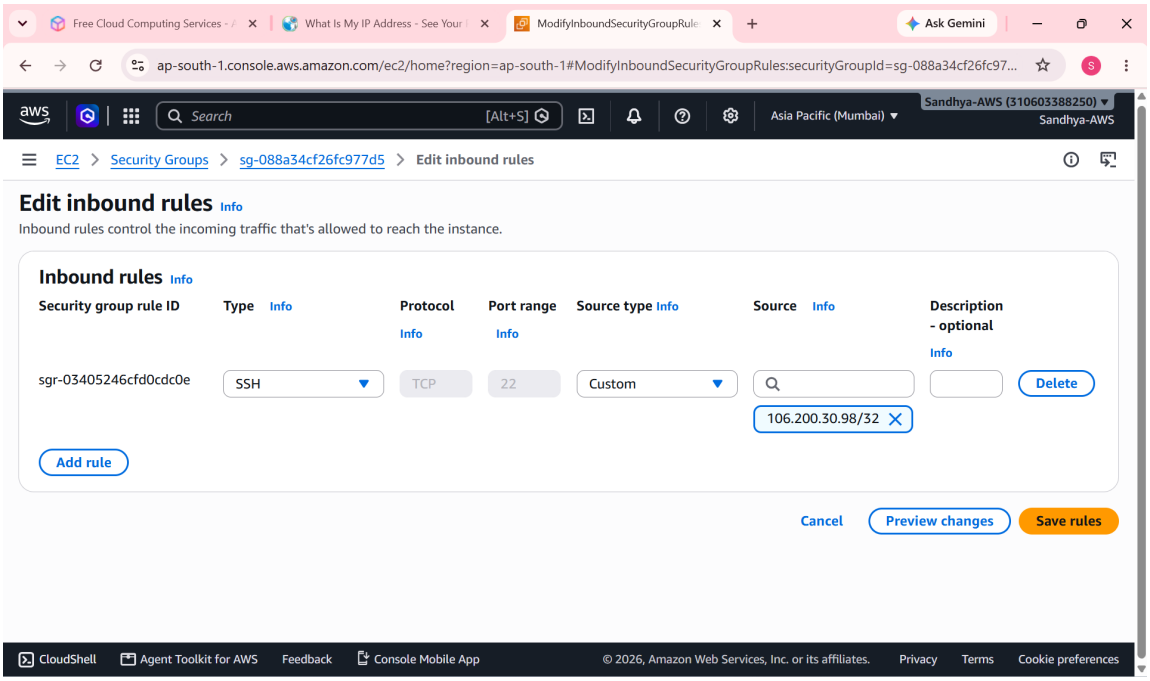
3. EC2 and Security Groups

The EC2 screenshot shows ewb-app-a, ewb-bastion, and ewb-app-b Running with 3/3 status checks passed.

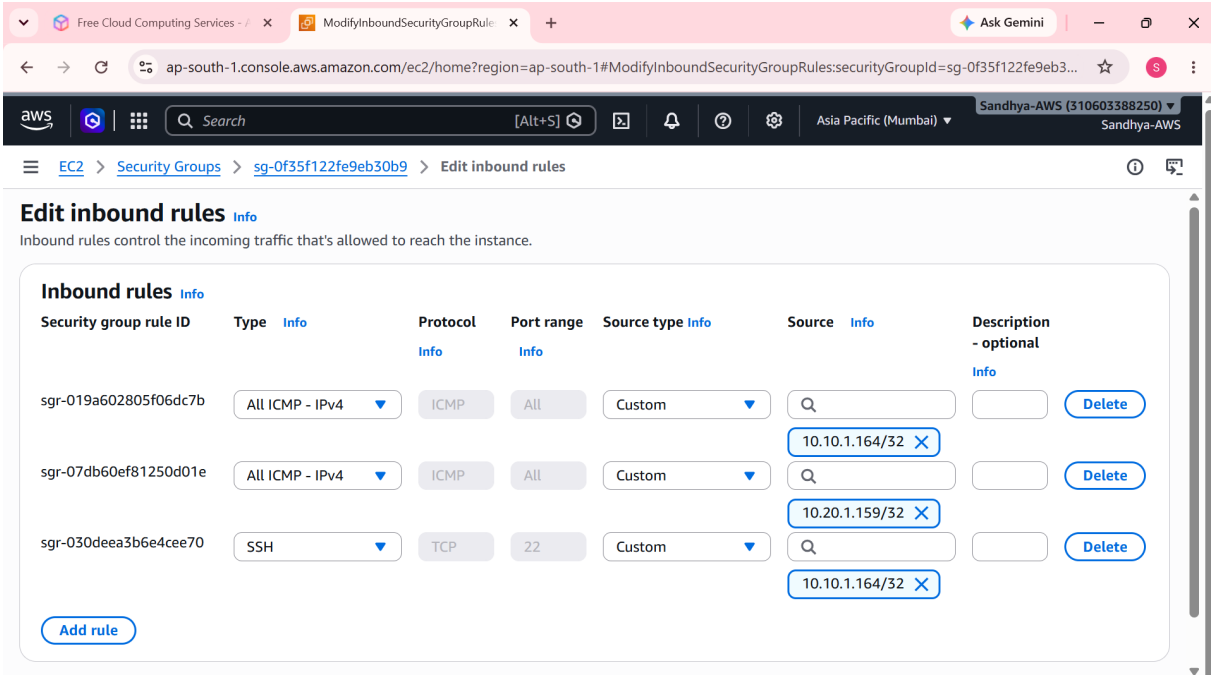
Evidence 8 — EC2 instance state and status checks.



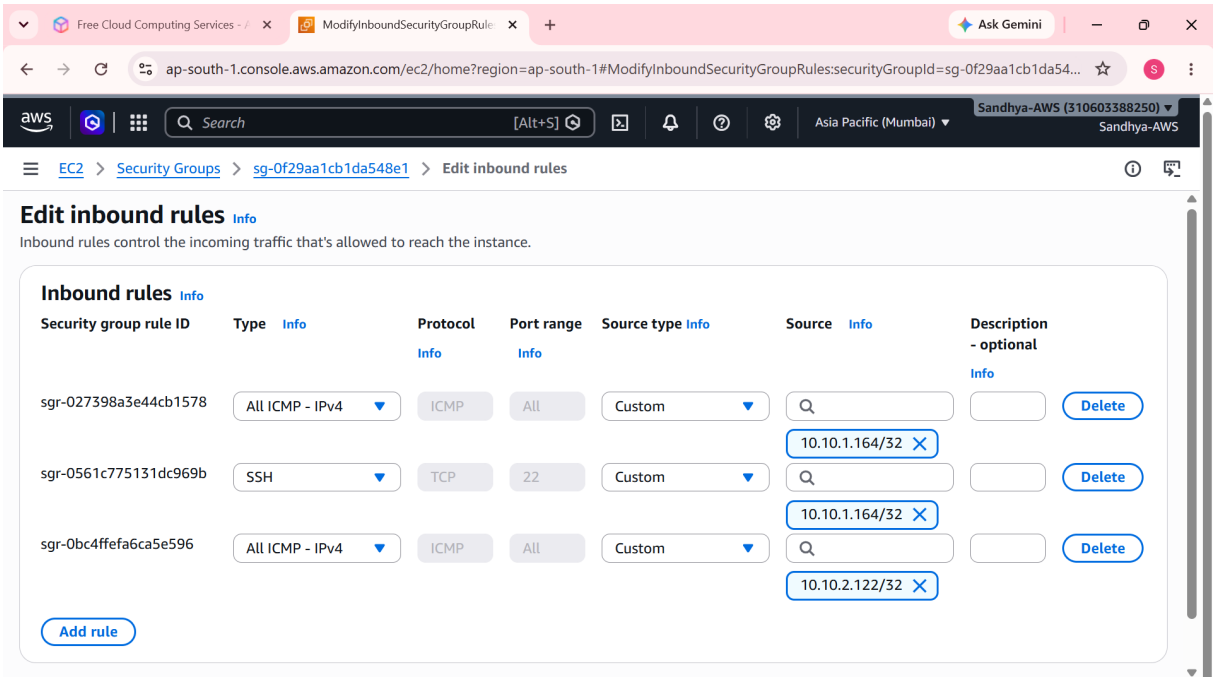
Evidence 9 — Bastion inbound rule.



Evidence 10 — App-A inbound rules.



Evidence 11 — App-B inbound rules.

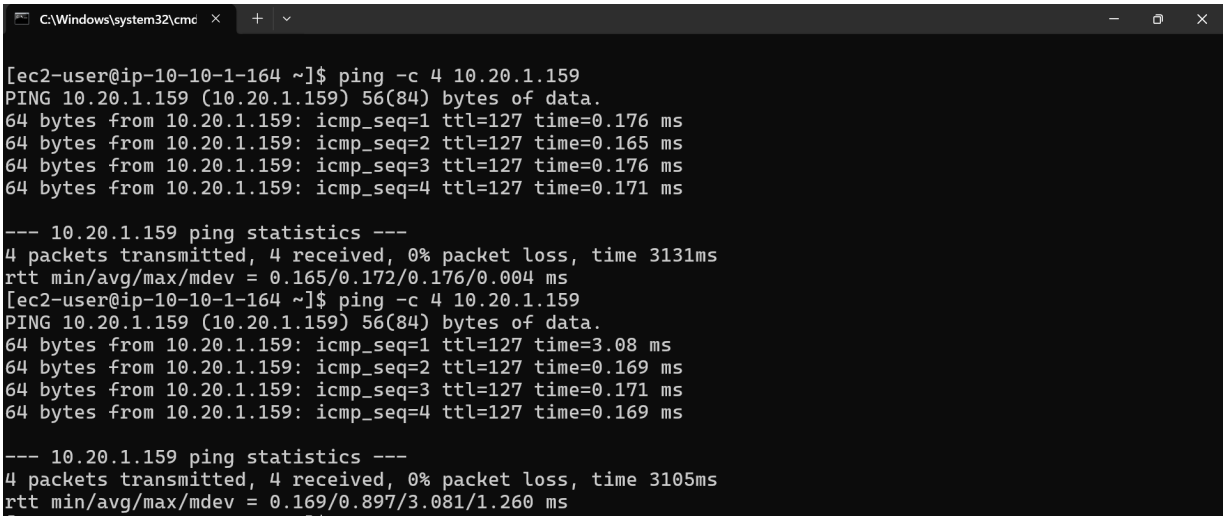


4. Connectivity Tests

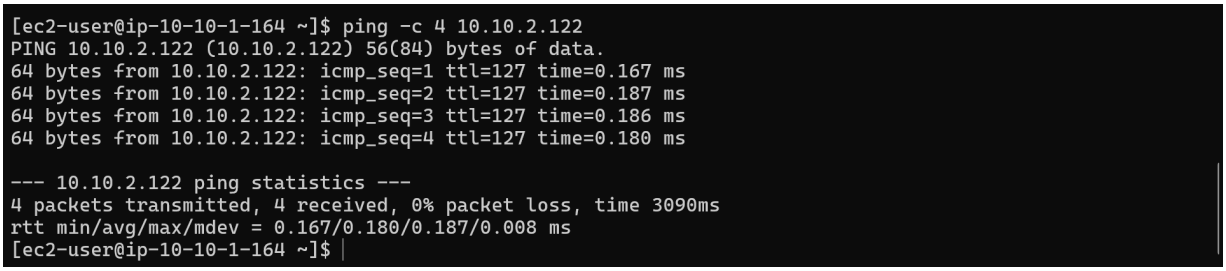
Forward path: App-A (10.10.2.122) sends ICMP echo requests to App-B (10.20.1.159). App-A's route table sends traffic for 10.20.0.0/16 through the VPC peering connection; App-B's security group must allow ICMP from App-A. Return path: App-B routes traffic for 10.10.0.0/16 through the peering connection to return replies to App-A.

The uploaded terminal screenshots include successful ping output, a failed ping, and an SSH session on App-B. The failure screenshot shown is from the bastion host pinging App-B, so it is not labeled as a verified App-A controlled failure. The screenshots do not prove that a baseline failure was captured before peering was configured.

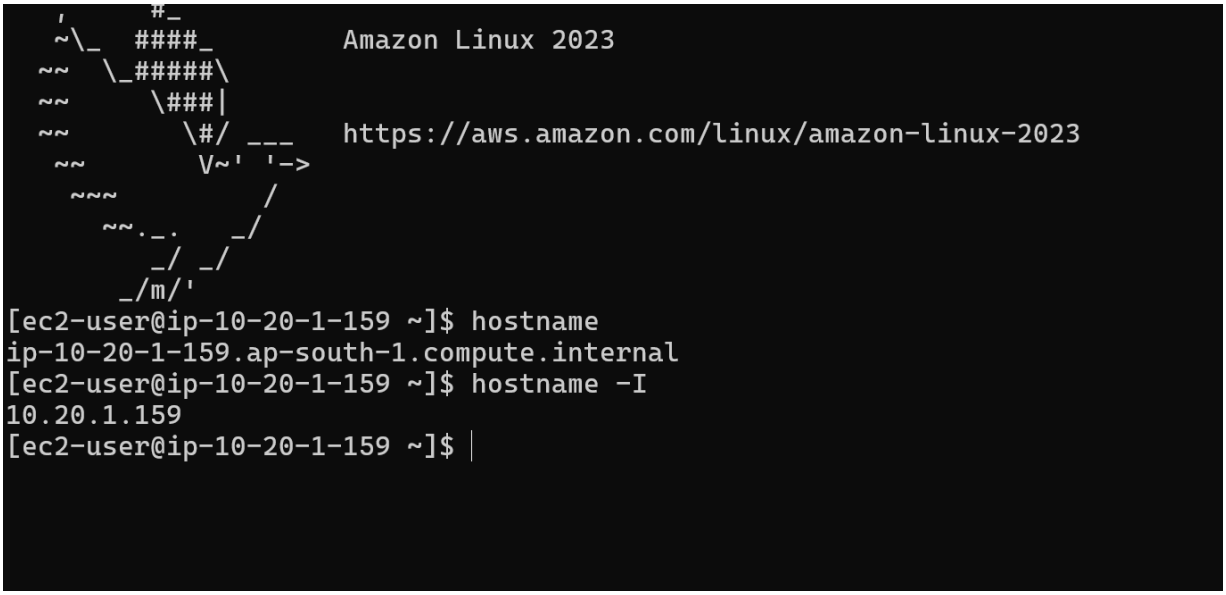
Evidence 12 — Forward ping screenshot; source host should be checked from prompt.



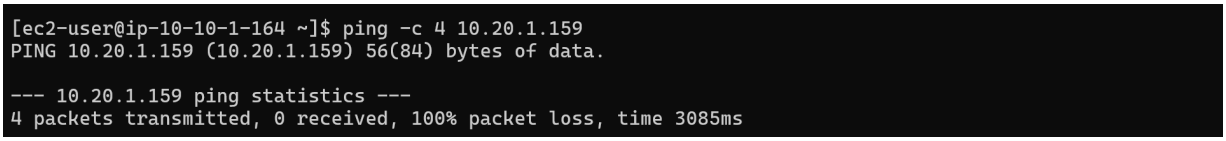
Evidence 13 — Reverse ping success.



Evidence 14 — SSH session on App-B through bastion; hostname and private IP visible.



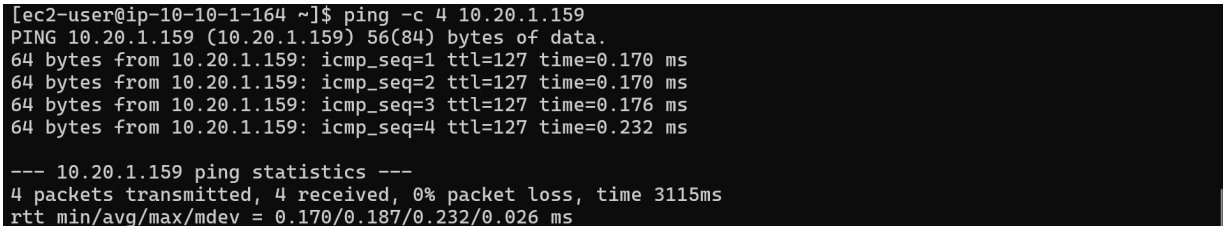
Evidence 15 — Bastion-to-App-B ping failure; not treated as App-A controlled failure.



```
[ec2-user@ip-10-10-1-164 ~]$ ping -c 4 10.20.1.159
PING 10.20.1.159 (10.20.1.159) 56(84) bytes of data.

--- 10.20.1.159 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3085ms
```

Evidence 16 — ICMP recovery ping success.



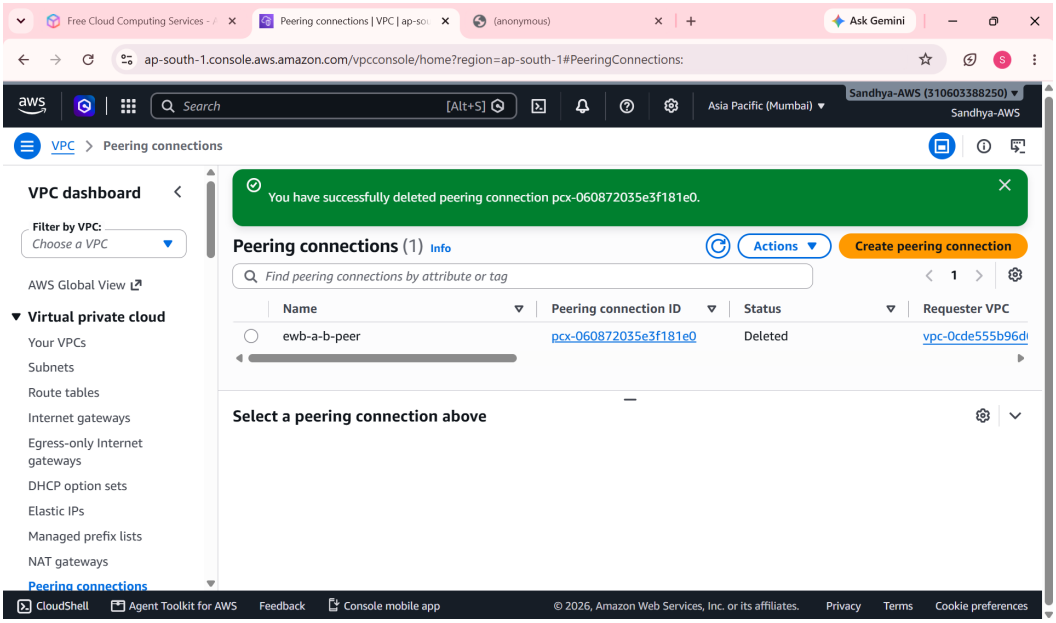
```
[ec2-user@ip-10-10-1-164 ~]$ ping -c 4 10.20.1.159
PING 10.20.1.159 (10.20.1.159) 56(84) bytes of data.
64 bytes from 10.20.1.159: icmp_seq=1 ttl=127 time=0.170 ms
64 bytes from 10.20.1.159: icmp_seq=2 ttl=127 time=0.170 ms
64 bytes from 10.20.1.159: icmp_seq=3 ttl=127 time=0.176 ms
64 bytes from 10.20.1.159: icmp_seq=4 ttl=127 time=0.232 ms

--- 10.20.1.159 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3115ms
rtt min/avg/max/mdev = 0.170/0.187/0.232/0.026 ms
```

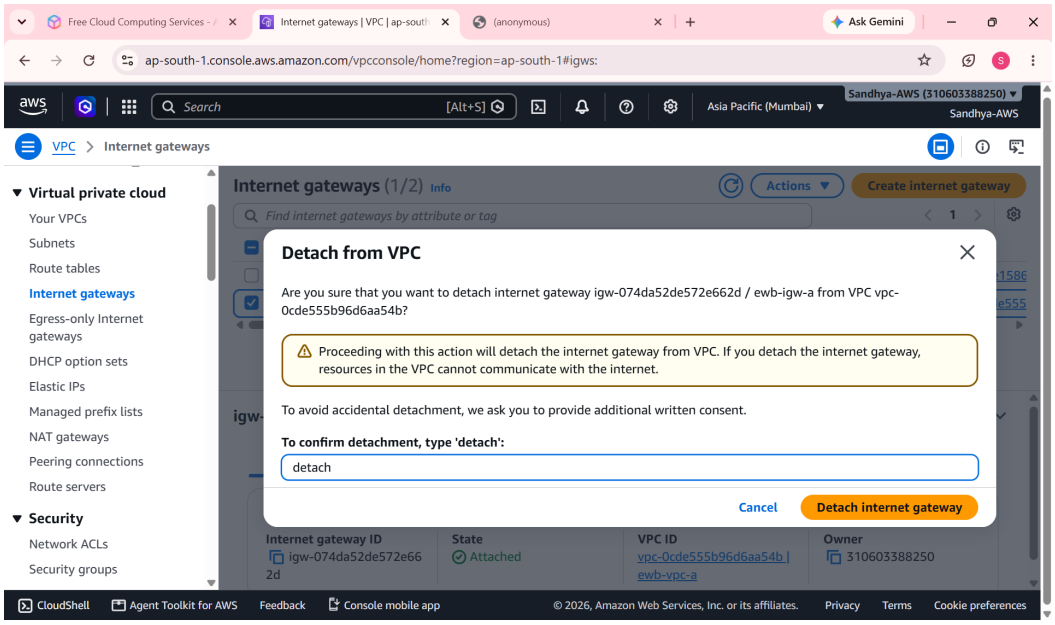
5. Cleanup and Learning

Cleanup evidence confirms the VPC peering connection was deleted and the Internet Gateway was detached. The gateway deletion confirmation screen and route-table deletion confirmation screen are also included. These screenshots do not prove that all EC2 instances, EBS volumes, security groups, subnets, and VPCs have been deleted. Verify the remaining resources in the AWS console before claiming complete cleanup.

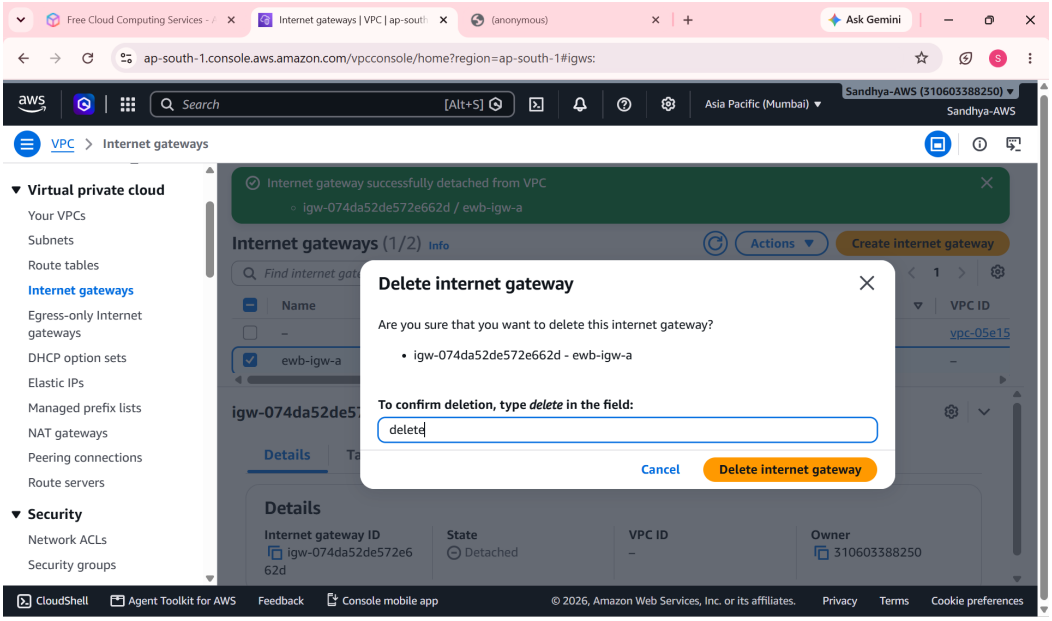
Evidence 17 — Peering deletion confirmed.



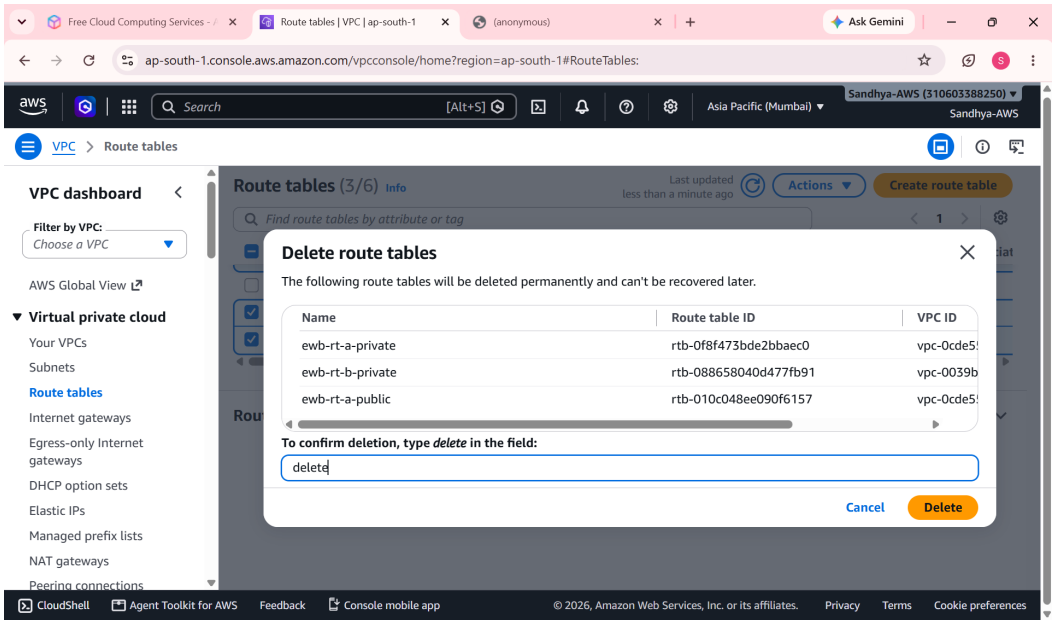
Evidence 18 — Internet Gateway detach confirmation.



Evidence 19 — Internet Gateway deletion confirmation screen.



Evidence 20 — Confirmation dialog to delete the three custom route tables.



What I learned

I learned how to create two VPCs with non-overlapping CIDR blocks, configure public and private subnets, set up route tables and VPC peering, restrict inbound traffic with security groups, test connectivity using ping, and connect to a private EC2 instance through a bastion host. I also learned to verify cleanup instead of assuming that deleting one network component removes every resource.

Security note: no private key, password, or access token is included in this report.

Evidence still needed for a fully complete submission: capture App-A → App-B ping success with the App-A prompt visible; controlled ICMP failure and recovery specifically from App-A after removing/restoring App-B's ICMP rule from 10.10.2.122/32; and final cleanup evidence showing all lab resources removed. Do not mislabel the current bastion ping failure as an App-A controlled failure.